

МЕТОДИЧЕСКИЕ УКАЗАНИЯ ПРЕПОДАВАТЕЛЮ

по проектной работе дисциплины «Технологии хранения и обработки больших данных»

1. Организация проектной работы

1.1. Формирование команд

Оптимальный размер команды – **3-4 человека**.

Формирование команд может быть **свободным** (по желанию студентов) или **по распределению** (для баланса сильных и слабых студентов).

В каждой команде рекомендуется выделить, ответственного за координацию и итоговый результат.

1.2. Выбор тем

Предложите студентам **список тем** с кратким описанием. Разрешите командам **предлагать свои темы** (при условии согласования).

Предложите студентам, используя **генеративный искусственный интеллект** (ChatGPT, YandexGPT, GigaChat), провести анализ тем и выбрать решаемую задачу по проектной работе, которая удовлетворяет требованиям проекта и дисциплине. Все запросы и предложения оформить в отчете в виде ссылки на чат с искусственным интеллектом (ИИ) и описанием выбранной задачи для реализации проектной работы. Убедитесь, что тема обеспечивает охват всех заявленных компетенций.

Критерии для выбора темы студентами:

- доступность данных (открытые наборы данных или **возможность генерации с помощью ИИ**);
- соответствие уровню сложности (не слишком простая и не слишком сложная);
- возможность применения всех изученных технологий.

1.3. Промежуточный контроль

Таблица 1. Рекомендуемая структура контроля

Неделя	Форма контроля	Что проверяется
Неделя 1	Утверждение темы и архитектуры	Корректность постановки задачи, выбор технологий
Неделя 3	Промежуточная демонстрация	Сбор и первичная подготовка данных
Неделя 5	Промежуточная демонстрация	Процесс преобразования данных, проверки качества, прогностическая модель
Неделя 6	Финальная защита	Полный проект

Рекомендация: на каждой контрольной точке проводить индивидуальные собеседования с каждым участником – это позволяет оценить понимание студентом своей части работы и его вклад.

Форматы промежуточного контроля:

- Краткая презентация (3-5 минут) с демонстрацией прогресса.
- Сдача промежуточного отчета.
- Индивидуальные консультации.

Предложите студентам подготовить финальную и промежуточные презентации с использованием ИИ (например, Кэмп AI, Presentsimple, Сократик, SlidePoint).

2. Рекомендации по оценке

Ключевые принципы:

1. **Прозрачность** – критерии оценивания должны быть известны студентам заранее (выдаются вместе с заданием).
2. **Объективность** – оценивается не только финальный результат, но и процесс.
3. **Командность** – учитывается вклад каждого участника.

Таблица 2. Типичные ошибки студентов и как на них указывать

Ошибка	Как указать
Нет обоснования выбора технологий	"В отчете отсутствует сравнительный анализ технологий. Почему вы выбрали ClickHouse, а не Greenplum? Почему Spark, а не Flink?"
Нет проверок качества данных	"Как вы гарантируете качество данных? Где проверки на пропуски, дубликаты, выбросы?"
Нет прослеживаемости данных	"Откуда взялись данные? Как они трансформировались? Где прослеживаемость?"
Нет обоснования задачи машинного обучения	"Почему это задача классификации, а не регрессии? Почему выбрали площадь под кривой (ROC-AUC), а не F1-меру?"
Архитектура не соответствует требованиям	"В задании было требование по масштабируемости. Как ваше решение масштабируется?"
Нет анализа влияния качества данных на модель	"Как повлияло качество данных на метрики модели? Что было бы с моделью без очистки данных?"

Бонусы и штрафы

Бонусы (до +5 баллов к итоговой оценке):

- использование дополнительных технологий (Flink, Kafka, Kubernetes);
- применение активного обучения для разметки данных;
- реализация продвинутого хранилища признаков;
- интеграция с большими языковыми моделями для естественно-языковых запросов;
- использование концепций управления данными.

Штрафы (до -5 баллов):

- нарушение сроков сдачи (без уважительной причины);
- плагиат – автоматически 0 баллов за соответствующий раздел;
- код не запускается на защите (без видео-демонстрации);
- отсутствие хотя бы одного обязательного компонента (Airflow, Spark, проверки качества данных).

3. Организация защиты

Рекомендуемый формат защиты:

1. Презентация (10-15 минут).
2. Демонстрация (5-10 минут):
 - контейнерная инфраструктура (вывод работающих контейнеров);
 - граф зависимостей задач в Airflow (успешно выполненные задачи);
 - данные в хранилищах (запросы);

- прослеживаемость данных (OpenMetadata);
 - прогностическая модель (метрики).
3. Вопросы (10-15 минут).

4. Материально-техническое обеспечение

Требования к аудитории для защиты:

- компьютер или ноутбук с установленным Docker, Git и Python;
- проектор или экран для презентации;
- доступ в интернет (для демонстрации облачных сервисов, интерфейсов прикладного программирования).

Доступ к вычислительным ресурсам:

1. Локальные компьютеры студентов (минимальные требования: 16 ГБ оперативной памяти, твердотельный накопитель SSD).
2. Лабораторные компьютеры (вузовский кластер).
3. Облачные платформы (Yandex Cloud, VK Cloud, SberCloud, Google Cloud).

Рекомендация: обеспечить возможность использования облачных платформ (бесплатные кредиты или гранты). Это позволит студентам работать с реальными объемами данных и демонстрировать масштабируемость.

5. Рекомендации по предотвращению плагиата

1. Индивидуальные темы для каждой команды. Команды работают над разными предметными областями или подзадачами.
2. Промежуточные контрольные точки. Регулярная проверка прогресса снижает возможность копирования чужого кода в конце семестра.
3. Проверка кода на плагиат. Используйте специализированные инструменты (Moss, JPlag).
4. Устные вопросы на защите. Индивидуальные вопросы по коду и архитектуре выявляют, насколько студент понимает свою работу.
5. Уникальные наборы данных. Если команды используют разные источники данных, копирование менее вероятно.

6. Примерный график консультаций

Таблица 3. График консультаций по неделям

Неделя	Консультация	Формат
Неделя 1	Утверждение тем, ответы на вопросы	Групповая (все команды)
Неделя 2	Проверка архитектуры, схемы хранения	Индивидуальная (по командам)
Неделя 3	Проверка сбора и подготовки данных	Индивидуальная (по командам)
Неделя 4	Проверка процесса преобразования данных	Индивидуальная (по командам)
Неделя 5	Проверка качества данных и прогностической модели	Индивидуальная (по командам)
Неделя 6	Предзащита, проверка отчетов	Индивидуальная (по командам)

7. Рекомендации по модернизации проекта

Таблица 4. Уровни сложности

Уровень	Описание	Требования
Базовый (3)	Базовая реализация всех этапов	Преобразование данных на Spark, Airflow, PostgreSQL, прогностическая модель
Средний (4)	Базовый и расширенные возможности	Добавить ClickHouse, прослеживаемость данных, проверки качества
Продвинутый (5)	Средний и инновации	Добавить потоковую обработку (Kafka/Flink), интерфейс больших языковых моделей, активное обучение

Дополнительные требования для продвинутых студентов

1. Реализация потоковой обработки – добавить Apache Kafka и Spark Streaming.
2. Интерфейс больших языковых моделей к данным – использовать интерфейс прикладного программирования больших языковых моделей (GigaChat, YandexGPT) для естественно-языковых запросов.
3. Хранилище признаков с версионированием – реализовать полноценное хранилище признаков с программным интерфейсом доступа.
4. Архитектура децентрализованного управления данными – разделить данные на домены, настроить децентрализованное управление.
5. Непрерывная интеграция и доставка для данных – настроить автоматическое тестирование и развертывание конвейеров.
6. Оркестрация в Kubernetes – вместо Docker Compose использовать Kubernetes.